

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : P. Abhiram

Reg. No : 11249A283

Class : II B.E. (CSE)

Section : 3

Course Code : BCSF244P90

Course Name : operating system.

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONA FIDE CERTIFICATE

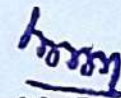
This is to certify that this is the bonafide record of work done by
Mr/Ms. P. Abhiram

with Reg. No 112494283 of II-B.E.(CSE) during the academic
year 2025 - 2026. sub-name: Operating system
sub-code:- BCSF 244890

Station: Enathur

Date: 25-04-26


Staff-in-charge


Head of the Department

Submitted for the Practical examination held on _____.

Examiner-1

Examiner-2

[illegible]

I N D E X

V. Abhishek
112490283
os lab, 53.

Name: XXXXXXXXXX Std.: Sec.:

Roll No.: Sub:

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
2)	13/2	Practice the following linux commands		
		a) file and directory related		
		b) Process and status information		
		c) Text Relation command. File permission		P800
3)	20/2	Execute a shell program:-		
		a) To find the whether a no is even or odd.		
		b) To find biggest of '2' no		
		c) To find biggest of '3' no		
		d) To find factorial of 'a' no		
		e) To display Fibonacci series		P800
4)	6/3	Practice the Linux Pipe, and filters		
		(a) Grep (b) Sed (c) Awk commands		P800
5)	13/3	Implement System calls using C		
		(a) stat (b) wait (c) (d) getpid		
		(e) opendir (f) open, read, write (g)		P800
6)	13/3	Implement Process management		
		(a) Fork (b) Exec (c)		P800
7)	20/3	Implement various CPU scheduling algorithms		
		(a) FIFO (b) Round Robin (c) SJF		P800
8)	3/4	Dead-lock using Banker's algorithm		P800
9)	3/4	Allocation method.		P800
		(a) First Fit (b) worst Fit		
		(c) Best Fit.		

Completed
~~P800~~
17/6/20

Practice the following Linux Commands

- File and directory related commands
- Process and status information commands
- Text related commands
- File permission commands

a) File and Directory Related Commands

These commands help you manage files and directories in the Linux file system

1. **ls** - List directory contents

Ex: **ls** → lists all files and directories in the current directory

2. **cd** - Change directory

Ex: **cd /home/user/documents** → navigates to documents

3. **pwd** - print working directory

Ex: **pwd** → shows the full path of your current directory

4. **mkdir** - Make a directory

Ex: **mkdir new_folder** → creates a new folder named new_folder

5. **rmdir** - remove a directory

Ex: **rmdir new_folder** → removes new_folder if it is empty.

6. **rm** - remove files or directories

7. **cp** - Copy files or directories

Ex: **cp files.txt /home/user/backup**

8. **mv** - move or rename files or directories

Ex: **mv file.txt newfile.txt** → rename file

9. touch - create an empty file or update the time stamp of an existing file

b. Process and status Information commands.

These commands provide information about running process, system status and resources.

1. ps: display current processes

Ex: ps → shows process running under your current user

2. top: Task manager (real-time view of system process)

Ex: Top → shows a realtime list of processes.

3. htop: An interactive version of top (requires installation)

4. kill: Kills a process PID

5. df: Disk space usage

Ex: df -h → shows disk space usage in human-readable format

6. free: memory usage

Ex: free -h → show memory usage in human-readable format

7. uptime: show how long the system has been running

8. who: show who logged into the system.

c. Text Related Commands.

These commands help you work with text files & output.

1. cat: Concatenate and display files.

Ex: cat file.txt → display content of file.txt

2. more: View the content of files page by page

Ex: more file.txt → views the content of file.txt

3. less: view the content of files with scrolling options.

Ex: less file.txt → similar more but with more advanced navigation

4. grep: search for txt with in file

Ex: grep "error" log.txt $\rightarrow$ search for the string "error" in log.txt

5. wc: word count

Ex: wc file.txt $\rightarrow$ shows the number of lines, words and characters in file.txt.

6. head: view the first few lines of a file

Ex: head -n 10 file.txt $\rightarrow$ shows the first 10 lines of file.txt.

7. tail: view the last few lines of a file.

8. sed: stream editor for text manipulation

Ex: sed 's/old/new/g' file.txt.

9. awk: pattern scanning and processing language

d. File Permission Commands.

These commands are used to manage file and directory permissions.

1. Chmod: change file permissions

Ex: chmod 755 files.txt $\rightarrow$ sets the file permission.

2. Chown: change file ownership

Ex: Chown user group file.txt $\rightarrow$ changes owner of file.txt to user.

3. Umask: set default file creation permissions.

Ex: Umask 022 $\rightarrow$ set the default permissions for new files.

4. stat: Display file or file system status

Ex: stat file.txt $\rightarrow$ shows detailed information.

Output

Enter a number: 9

9 is odd

Output :-

Enter first number: 7

Enter second number: 6

7 is the biggest

~~Output :-~~

~~Enter first number: 6~~

~~Enter second number: 8~~

~~Enter third number: 9~~

~~9 is the biggest~~

Result : The program were written and executed the terminal successfully

Execute a shell program

a) To find whether a number is even or odd

```
echo "enter a number;"
read num
if [ $(num%2) -eq 0 ]; then
    echo "$num is Even"
else
    echo "$num is odd"
fi
```

b) To find the Biggest of two numbers

```
echo "Enter first number:"
read num1
echo "Enter second number:"
read num2
if [ $num1 -gt $num2 ]; then
    echo "$num1 is the biggest"
elif [ $num1 -lt $num2 ]; then
    echo "$num2 is the biggest"
else
    echo "Both numbers are equal"
fi
```

c) To find the biggest of Three numbers

```
echo "Enter first number:"
read num1
echo "Enter second number:"
read num2
echo "Enter third number:"
read num3
if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ]; then
    echo "$num1 is the biggest"
```


Output:

Enter a number: 4

Factorial of n is 24

Output:

Enter the number of terms: 7

Fibonacci series: 0 1 1 2 3 5 8

Result:

The programs were written and executed in the terminal successfully.


```

elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ]; then
    echo " $num 2 is the biggest"
else
    echo " $num 3 is the biggest"
fi

```

d) To find the factorial of a number.

```

echo "Enter a number:"

```

```

read num

```

```

factorial = 1

```

```

for (i=1; i <= $num; i++)

```

```

do

```

```

    factorial = $ (factorial * i)

```

```

done

```

```

echo "Factorial of $num is $factorial"

```

e) To display Fibonacci Series.

```

echo "Enter the number of terms"

```

```

read terms

```

```

a=0

```

```

b=1

```

```

echo "Fibonacci Series:"

```

```

for (i=0; i < $terms; i++)

```

```

do

```

```

    echo -n " $a"

```

```

    next = $ (a+b)

```

```

    a = $b

```

```

    b = $next

```

```

done

```

```

echo

```


Output:

Enter the file to search

one.txt

Enter the search term (i)

i

hi

i

Enter file to search

one.txt

enter old word

hi

enter new word

hi

The word hi is replaced by hi in one.txt

enter the file name

even.sh

echo a

read

i = \$1

echo is

else

echo is

fi

else -> Row

Column

Result:

The program were written and
executed the terminal successfully

Practice the linux pipes and filters commands.a) Grep:

Search for a specific word ("error") in a log file and display the matching lines

Code:

```
echo "Enter the log file to search:"
read file name
echo "Enter the search term:"
read search-term
grep "$search-term" "$file name"
```

b) Sed:

Replace occurrences of a word in a file and save the changes back to the file

code:

```
echo "enter file to search"
read file
echo "enter old-word"
read old-word
echo "enter new-word"
read new-word
sed -i "s/$old-word/$new-word/g" "$file name"
echo "The word '$old-word' has been replaced with '$new-word' in $file name"
```

c) awk:

print the first and third column of a file where columns are separated by spaces.

Code: echo "Enter the file name:"

read file name

awk '{print \$1, \$3}' "\$file name"

awk 'NR==5' "\$file name"

Result:

The program were written and executed the terminal successfully

Output:

Usage: ./output <text>

/output text.txt

File size: 238 bytes

Number of links: 1

File inode: 14567999

File permissions: 664

Output:

child process executing

child process exiting

parent process resumed after child
termination.

Implement system calls using C

a) stat()

purpose: Get information about a file

code

```

#include <stdio.h>
#include <sys/stat.h>
int main (int argc, char *argv[])
{
    struct stat filestat;
    if (argc < 2) {
        printf("usage: %s < filename> \n", argv[0]);
        return 1;
    }
    if (stat (argv[1], & filestat) == -1) {
        perror("stat");
        return 1;
    }
    printf("File size: %ld bytes \n", filestat.st_size);
    printf("Number of links: %ld \n", filestat.st_nlink);
    printf("File inode: %ld \n", filestat.st_ino);
    printf("File permissions: %ld \n", filestat.st_mode
        & 0777);
    return 0;
}

```

b) wait()

purpose: parent process waits for child process to finish

program:-

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/wait.h>

```


Implement system calls using

(2) space ()

process, get information about a file

Output:-

process id: 3176

parent process id: 3136

(code)

int main (int argc, char *argv[])

struct stat file_stat;

if (argc < 2)

Print ("Usage: ./a.out filename\n");

return 1;

}

if (stat (argv[1], &file_stat) != 0)

error ("stat");

return 1;

}

Print ("File size: %ld bytes\n", file_stat.st_size);

Print ("Number of links: %ld\n", file_stat.st_nlink);

Print ("File mode: %ld\n", file_stat.st_mode);

Print ("File permissions: %ld\n", file_stat.st_perm);

0

return 0;

}

(code)

Process, parent process id, file, permissions, file size

Process

File size, file mode, file permissions

File size, file mode, file permissions


```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
pid_t pid;
```

```
pid = fork();
```

```
if (pid == 0) {
```

```
printf("child process executing\n");
```

```
sleep(2);
```

```
printf("child process exiting\n");
```

```
}
```

```
else {
```

```
wait(NULL);
```

```
printf("parent process resumed after child  
termination\n");
```

```
}
```

```
return 0;
```

```
}
```

(c) get pid() system call

purpose:

Get process ID of current process

code:-

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
printf("process ID: %d\n", getpid());
```

```
printf("parent process ID: %d\n", getppid());
```

```
return 0;
```

```
}
```


output:

mkdir 296

contents of directory;

Result:

The program were written and executed
be ~~terminal~~ successfully.

d) opendir() and readdir() system calls.

purpose: Read contents of a directory

code:

```
#include <stdio.h>
#include <dirent.h>
int main (int argc, char * argv[])
{
    DIR * dir;
    struct dirent * entry;
    if (argc < 2) {
        printf("usage: %s <directory path> \n", argv[0]);
        return 1;
    }
    dir = opendir (argv[1]);
    if (dir == NULL) {
        perror("opendir");
        return 1;
    }
    printf("Contents of directory: \n");
    while ((entry = readdir (dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir (dir);
    return 0;
}
```


Output:

Parent process;

parent PID: 3553

child PID: 3554

child process;

child PID: 3554

→ parent PID: 3553

Implement Process Management.Q) Fork () :

purpose: Create a new process (child) from an existing process

Code:

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0) {
        printf("Fork Failed\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("parent PID: %d\n", getppid());
    }
    else {
        printf("parent process\n");
        printf("parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
    }
    return 0;
}
```


Output

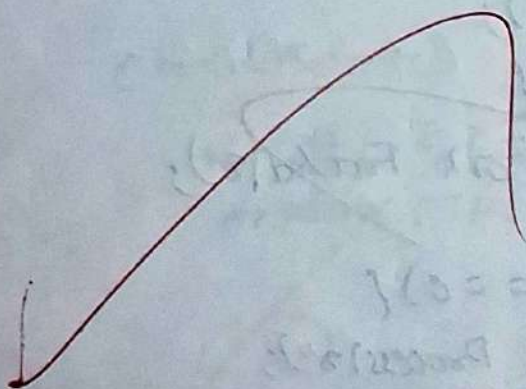
Parent process

child process before exec()

291 exp 3b.5h exp 3d.5h exp 4.5h

ex 5b.c ex 5c.c exp 6b.c

text -txt.



Result:-

The program were written and executed the terminal successfully

Explanation:-

Fork() returns;

* 0 $\rightarrow$ child process

* > 0 $\rightarrow$ parent process (child PID)

* < 0 $\rightarrow$ error.

b) exec():

purpose: Replace the current process image with a new program

code:-

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;
    pid = fork();
    if (pid == 0) {
        printf("child process before exec()\n");
        exec("/bin/ls", "ls", NULL);
        printf("This line will not be executed\n");
    }
    else {
        printf("parent process\n");
    }
    return 0;
}
```


Output:

Enter number of processes: 4

Enter burst times:

1

2

3

4

Process	BT	WT	TAT
P ₁	1	0	1
P ₂	2	1	3
P ₃	3	3	6
P ₄	4	6	10

Average waiting time = 2.50

Average Turnaround time = 5.00

Implement various cpu scheduling algorithmsQ) FIFO / FCFS (First come First serve)Purpose:

- * Processes executes in order of arrival
- * Non-preemptive

Code:-

```

#include <stdio.h>
int main()
{
    int n, i;
    int bt[10], wt[10], tat[10];
    float avg_wt = 0, avg_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter burst times: \n");
    for (i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
    }
    wt[0] = 0;
    for (i = 0; i < n; i++) {
        wt[i] = wt[i-1] + bt[i-1];
    }
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
        avg_wt += wt[i];
        avg_tat += tat[i];
    }
    printf("\n Process | EBT | WT | TAT | \n");
    for (i = 0; i < n; i++) {
        printf("P%d | %d | %d | %d | \n", i+1, bt[i], wt[i], tat[i]);
    }
}

```



```

print f("Average waiting Time = %.2f\n", avg-wt);
print f("Average Turnaround Time = %.2f\n", avg-tat/n);
return 0;
}

```

Enter number of processes: 5

Enter burst times: 11 12 13 14 15

Process	BT	WT	TAT
P1	11	00	11
P2	12	11	23
P3	13	23	36
P4	14	36	50
P5	15	50	65

Average waiting time = 21.00

Average turnaround time = 37.00

Result:

The program was written and executed in the terminal successfully

Output:-

Enter number of processes: 5

Enter burst times:

- 11
- 12
- 13
- 14
- 15

Enter time quantum: 5

process	BT	WT	TAT
P1	11	40	51
P2	12	41	53
P3	13	43	56
P4	14	46	60
P5	15	50	65

Average waiting time = 44.00

Average Turn around Time = 57.00

b) Round Robin Scheduling

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int n, i, tq;
```

```
    int bt[10], rt[10], wt[10], tat[10];
```

```
    int time = 0, done;
```

```
    float avg-wt = 0, avg-tat = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter burst times: \n");
```

```
    for (i=0; i<n; i++) {
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
        wt[i] = 0;
```

```
    }
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &tq);
```

```
do {
```

```
    done = 1;
```

```
    for (i=0; i<n; i++) {
```

```
        if (rt[i] > 0) {
```

```
            done = 0;
```

```
            if (rt[i] > tq) {
```

```
                time += tq;
```

```
                rt[i] -= tq;
```

```
            } else {
```

```
                time += rt[i];
```

```
                wt[i] = time - bt[i];
```

```
                rt[i] = 0;
```


}

}

} while (!done);

```
for (i=0; i<n; i++) {  
    tat[i] = bt[i] + wt[i];  
    avg_wt += wt[i];  
    avg_tat += tat[i];  
}
```

```
printf("\n process | tB | tW | tT | n");  
for (i=0; i<n; i++) {  
    printf("P%d | t %d | t %d | n", i+1, bt[i], wt[i],  
        tat[i]);  
}
```

```
printf("Average waiting Time = %.2f\n", avg_wt/n);  
printf("Average Turnaround Time = %.2f\n", avg_tat/n);  
return 0;  
}
```


Output :-

Enter number of processes: 4

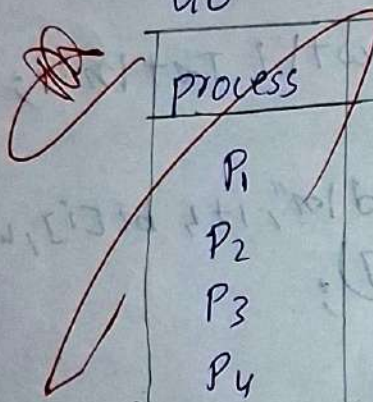
Enter burst time:

10

20

30

40



process	BT	WT	TAT
P ₁	10	0	10
P ₂	20	10	30
P ₃	30	30	60
P ₄	40	60	100

Average waiting Time = 25.00

Average Turnaround Time = 50.00

C) SIF (Shortest Job First):-

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int n, i, j;  
    int bt[10], wt[10], tat[10], temp;  
    float avg-wt=0, avg-tat=0;  
    printf("Enter number of processes:");  
    scanf("%d", &bt n)
```

```
    printf("Enter burst time:\n");
```

```
    for(i=0; i<n; i++) {
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    for(i=0; i<n-1; i++) {
```

```
        for(j=i+1; j<n; j++) {
```

```
            if(bt[i] > bt[j]) {
```

```
                temp = bt[i];
```

```
                bt[i] = bt[j];
```

```
                bt[j] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
    wt[0] = 0;
```

```
    for(i=1; i<n; i++) {
```

```
        wt[i] = wt[i-1] + bt[i-1];
```

```
    }
```

```
    for(i=0; i<n; i++) {
```

```
        tat[i] = bt[i] + wt[i];
```

```
        avg-wt += wt[i];
```

```
        avg-tat += tat[i];
```



```

}
printf("\n Process | BT | WT | TT |n");
for(i=0; i<n; i++){
    printf("P %d | b %d | t %d | t %d |n", i+1, bt[i], wt[i],
        tat[i]);
}
printf("Average waiting Time = %d |n", avg-wt/n);
printf("Average Turnaround Time = %d |n", avg-tat/n);
return 0;
}

```

Result

The program was written and executed in the terminal successfully.

Output:

Enter number of processes: 3

Enter number of resources: 2

Enter Allocation Matrix:

2

1

3

4

5

6

Enter maximum matrix:

4

1

2

3

5

6

Enter Available Resources

6

5

System is in safe state

Safe sequence: P0 P1 P2

Ex-8

Write C program to avoid deadlock using

Banker's Algorithm

Aim: To check whether the system is in a safe state and to find a safe sequence using Banker's Algorithm.

Program:

```
#include <stdio.h>
int main()
{
    int m, n, i, j, k;
    int alloc[10][10], max[10][10], need[10][10],
    int avail[10], finish[10], safe seq[10],
int avail
    printf("Enter number of processes:");
    scanf("%d", &n);
    printf("Enter number of resources:");
    scanf("%d", &m);
    printf("Enter Allocation Matrix: \n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter maximum Matrix: \n");
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available Resources: \n");
    for (i=0; i<m; i++)
        scanf("%d", &avail[i]);
```



```
for (i=0; i<n; i++)
```

```
for (j=0; j<m; j++)
```

```
need[i][j] = max[i][j] - alloc[i][j];
```

```
for (i=0; i<n; i++)
```

```
finish[i] = 0;
```

```
in count = 0;
```

```
while (count < n) {
```

```
int found = 0;
```

```
for (i=0; i<n; i++) {
```

```
if (finish[i] == 0) {
```

```
for (j=0; j<m; j++)
```

```
if (need[i][j] > avail[j])
```

```
break;
```

```
if (j == m) {
```

```
for (k=0; k<m; k++)
```

```
avail[k] += alloc[i][k];
```

```
safe seq [count++] = i;
```

```
finish[i] = 1;
```

```
found = 1;
```

```
}
```

```
}
```

```
}
```

```
if (!found) {
```

```
printf("The system is in UNSAFE state (Dead lock possible)");
```

```
return 0;
```

```
}
```

```
}
```

```
printf("The system is in SAFE state\n safe sequence:");
```

```
for (i=0; i<n; i++)
```

```
printf("P%d ", safe seq[i]);
```

```
printf("\n");
```

```
return 0;
```

```
}
```


Ex: 2
Bubble Sort Algorithm
To check whether the array is sorted or not, we use a flag variable. If the array is sorted, the flag will be 1, otherwise it will be 0.

Program:

```
#include <iostream>
using namespace std;
int main()
{
    int arr[10];
    cout << "Enter 10 numbers: ";
    for (int i = 0; i < 10; i++)
    {
        cin >> arr[i];
    }
    int flag = 1;
    for (int i = 0; i < 10; i++)
    {
        for (int j = i + 1; j < 10; j++)
        {
            if (arr[i] > arr[j])
            {
                swap(arr[i], arr[j]);
            }
        }
    }
    cout << "Sorted array: ";
    for (int i = 0; i < 10; i++)
    {
        cout << arr[i] << " ";
    }
    return 0;
}
```



Result:

The program was written and executed the terminal successfully

Ex. No: 9

Memory Allocation Method

- First Fit
- Worst Fit
- Best Fit

a) First Fit

allocates the first block that is large

enough.

Program

```
#include <stdio.h>
int main()
{
    int nb, np, i, j;
    int b[10], p[10], alloc[10];
    printf("Enter number of blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (i = 0; i < nb; i++)
        scanf("%d", &b[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++)
        scanf("%d", &p[i]);
    for (i = 0; i < np; i++)
        alloc[i] = -1;
    for (i = 0; i < np; i++) {
        for (j = 0; j < nb; j++) {
            if (b[j] >= p[i]) {
                alloc[i] = j;
                b[j] -= p[i];
                break;
            }
        }
        printf("Process %d size %d Block %d", i, p[i], alloc[i]);
        for (i = 0; i < np; i++) {
            if (alloc[i] != -1)
```


Output

Enter number of blocks: 4

Enter size of each block:

4

5

2

1

Enter number of processes: 3

Enter size of each process:

2

1

4

Process	size	Block
P ₁	2	1
P ₂	1	1
P ₃	4	2


```

    printf("P%d | T%d | Not allocated | m", i+1, P[i]);
    else
        printf("P%d | T%d | Not allocated | m", i+1, P[i]);
    }
    return 0;
}

```

b) worst fit Algorithm;

allocates the largest available block

Program;

```

#include <stdio.h>
int main()
{
    int nb, np, i, j;
    int b[10], p[10], alloc[10];
    int max, pos;
    printf("Enter number of blocks:");
    scanf("%d", &nb);
    printf("Enter size of each block: m");
    for (i=0; i<nb; i++)
        scanf("%d", &np);
    printf("Enter size of each process: m");
    for (i=0; i<np; i++)
        scanf("%d", &p[i]);
    for (i=0; i<np; i++)
        alloc[i] = -1;
    for (i=0; i<np; i++) {
        max = -1;
        pos = -1;
        for (j=0; j<nb; j++) {
            if (b[j] >= p[i] && b[j] > max) {
                max = b[j];
                pos = j;
            }
        }
        if (pos != -1) {
            alloc[i] = pos;
            b[pos] -= p[i];
        }
    }
}

```


Output

Enter number of blocks: 4

Enter size of each block:

4

5

2

1

Enter

"

processes: 3

process:

2

1

4

Process	size	Block
P ₁	2	2
P ₂	1	1
P ₃	4	not allocated


```

}
}
printf("Process | size | Block |");
for (i=0; i < np; i++) {
    if (alloc[i] != -1)
        printf("P %d | %d | %d |", i+1, p[i], alloc[i]);
    else
        printf("P %d | %d | Not allocated |", i+1, p[i]);
}
return 0;
}

```

c) Best fit

allocates the smallest block that is sufficient.

Program:

```

#include <stdio.h>
int main()
{
    int mb, np, i, j;
    int b[10], p[10], alloc[10];
    int min, pos;
    printf("Enter number of blocks:");
    scanf("%d", &mb);
    printf("Enter size of each block:");
    for (i=0; i < mb; i++)
        scanf("%d", &b[i]);
    printf("Enter number of processes:");
    scanf("%d", &np);
    printf("Enter size of each process:");
    for (i=0; i < np; i++)
        scanf("%d", &p[i]);
}

```



```
for (i=0; i<nP; i++)
```

```
scanf alloc[i] = -1
```

```
for (i=0; i<nP; i++) {
```

```
    min = 9999;
```

```
    pos = -1;
```

```
    for (j=0; j<nB; j++) {
```

```
        if (b[j] == P[i] && b[j] < min) {
```

```
            min = b[j];
```

```
            pos = j;
```

```
        }
```

```
    }
```

```
    if (pos != -1) {
```

```
        alloc[i] = pos;
```

```
        b[pos] = P[i];
```

```
    }
```

```
}
```

```
printf("In Process) {size} Block\n");
```

```
for (i=0; i<nP; i++) {
```

```
    if (alloc[i] != -1)
```

```
        printf("P%d) %d) %d)\n", i+1, P[i], alloc[i]+1);
```

```
    else
```

```
        printf("P%d) %d) NOT Allocated)\n", i+1, P[i]);
```

```
}
```

```
return 0;
```

```
}
```

Result:

The program was written and executed the terminal successfully.

Output

Enter number of blocks: 4

Enter size of each block:

4

5

6

1

Enter number of processes: 4

Enter size of each process:

6

5

4

2

Process	Size	Block
P ₁	6	3
P ₂	5	2
P ₃	4	1
P ₄	2	not allocated

Completed

~~2000~~
17/4/20